POSHA

Systems Engineering Internship Assignment

Kunal Paroda

Roll No: 24B0071

2nd Year, B.Tech. Aerospace Engineering
Indian Institute of Technology Bombay

Submitted to: POSHA Engineering Team
Submission Date: April 9, 2026

Assignment Overview

Task 1:	Servo Motor QC Jig Design
Task 2:	Servo System Reliability & Safety Analysis
Task 3:	POSHA System Design Improvements

Contents

1 Task 1: Servo Motor QC Jig Design **4**

1.1 Motor Selection and Specification Baseline 4

1.1.1 Dynamixel XL430-W250-T: The Real Specification Baseline 4

1.1.2 What is a Smart Actuator vs. a Hobby Servo? 6

1.2 Concept 1: FOC-based Autonomous Sweep (Considered and Set Aside) . . . 6

1.3 Measurement Strategy 8

1.3.1 Problem Decomposition 8

1.3.2 Load Mechanism and Torque Control 8

1.3.3 Torque Measurement: Dual Load Cell 11

1.3.4 Speed and Position Measurement 11

1.3.5 Open Question: Stall Current Inflation 11

1.4 Structural Design: Aluminium 2020 Extrusion Frame 11

1.4.1 Why Aluminium 2020 Over Steel DIN Rail 11

1.5 Mechanical Design: Components and Assembly 12

1.5.1 Custom Servo Mount and Output Horn 12

1.5.2 Gearbox and Split-Half Design 13

1.5.3 Encoder Coupling 13

1.6 CAD Model and Assembly Simulation 13

1.7 Electronics: Sensor Integration 15

1.7.1 Overview of Sensor Choices 15

1.7.2 Why Arduino Over Pico or ESP32 16

1.7.3 Wiring: HX711 Load Cells (Both Channels) 16

1.7.4 Wiring: ADS1115 ADC 17

1.7.5 Wiring: INA219 Servo Current Sensor 17

1.7.6 Wiring: AS5600 Encoder 18
1.7.7 Wiring: Servo and DC Motor 19
1.8 Schematic Diagrams 19
1.9 PCB Design 20
1.9.1 Note on PCB Completion Status 20
1.9.2 Module-based Approach 20
1.9.3 PCB IC Selection 20
1.10 Bill of Materials 21
1.10.1 PCB Version: Changed Items 21
1.11 Firmware 22
1.11.1 Module-based Code (Arduino Uno, all breakout modules) 22
1.11.2 PCB-based Code (ATmega328P with discrete ICs) 25
1.11.3 DC Motor Torque Control Loop Explained 28
1.12 Calibration 28
1.12.1 Load Cell Calibration 28
1.12.2 Linearity Verification 29
1.12.3 Encoder Verification 29
1.13 Maintenance and Usability 29

2 Task 2: Servo System Reliability and Safety 30

2.1 How Servo Motors Can Fail 30
2.1.1 The Position Sensor Wears Out 30
2.1.2 Running at Stall for Too Long Burns the Motor 30
2.1.3 The Gears Wear and Develop Backlash 31
2.1.4 Electrical Noise on the Signal Wire 31
2.1.5 Voltage Droop When Everything Turns On at Once 32

2.2 Deep Dive: Gear Wear and PWM Noise 32

2.2.1 Gear Wear Under Repeated Near-Stall Loading 32

2.2.2 PWM Noise in a Motor-Rich Environment 32

2.3 Debugging Process 33

2.4 Improvement Suggestions 34

3 Task 3: POSHA System Design Improvements 34

3.1 Context and Background 34

3.2 Reducing Total Cooking Time 34

3.2.1 Temperature Overshoot for Smooth Phase Transitions 34

3.2.2 Pre-heating Before the User Presses Start 35

3.2.3 Overlapping Dispense and Heating 35

3.2.4 Torque-based Stirring Optimisation 35

3.3 Design and Functionality Improvements 35

3.3.1 Spice Dispenser Architecture 35

3.3.2 Accessibility for the End User 36

3.3.3 Adaptive Recipe Learning 36

1. Task 1: Servo Motor QC Jig Design

1.1 Motor Selection and Specification Baseline

No specific servo was called out in the brief, so I started by looking at the full range of servo motors that could realistically appear in a robot like POSHA: standard hobby PWM servos, and the more advanced smart actuator type like the Dynamixel series. The selection criteria were global off-the-shelf availability, complete manufacturer documentation, and voltage compatibility with small-robot power budgets.

Important note on servo type used in this jig: The QC jig hardware (load cell, gearbox, encoder, H-bridge, and all electronics) was designed and built around the DS3218 as a representative test candidate. The specifications and performance graph in the next table match the DS3218. The Dynamixel XL430-W250-T has numerically identical headline specs (same stall torque, same speed, same supply range), so the same jig covers both. For production use on the XL430, only the servo mount and output horn adapter need to change to suit Dynamixel's proprietary horn spline.

1.1.1 Dynamixel XL430-W250-T: The Real Specification Baseline

Since POSHA is a production robot and almost certainly uses smart actuators, the correct device to characterise is the Dynamixel XL430-W250-T. Its specifications are reproduced below from the official Robotis documentation.

Table 1: Dynamixel XL430-W250-T (2018) full specification

Item	**Specification**
MCU	ARM Cortex-M3, 72 MHz, 32-bit
Position sensor	Contactless absolute encoder, 12-bit, 360 (AMS AS5601)
Motor type	Cored DC brush motor
Gear ratio	258.5 : 1
Stall torque	1.0 N m at 9.0 V / 1.0 A 1.4 N m at 11.1 V / 1.3 A 1.5 N m at 12.0 V / 1.4 A
No-load speed	47 rpm at 9.0 V 57 rpm at 11.1 V 61 rpm at 12.0 V
Standby current	52 mA
Resolution	4096 pulse/rev (0.088 per count)
Operating modes	Velocity / Position (0–360) / Extended position (multi-turn) / PWM voltage
Communication	TTL Half-Duplex, 9600 bps to 4.5 Mbps
Physical connection	TTL Multidrop Bus, 5 V logic
ID range	0–252 (253 total IDs)
Feedback	Position, velocity, load, temperature, input voltage, realtime tick, trajectory
Operating temperature	-5 to $+72$ °C
Input voltage	6.5–12.0 V (recommended 11.1 V, 3S Li-ion)
Weight	57.2 g
Dimensions	$28.5 \times 46.5 \times 34$ mm
Case and gear material	Engineering plastic

What does Kt mean? The XL430 datasheet gives $K_t = 1.000\,\mathrm{N\,m/A}$ at 9 V and approximately $1.075\,\mathrm{N\,m/A}$ at 11.1 V. K_t is the torque constant: it tells you how much torque the motor produces per ampere of current. If you know the current and you know K_t, you can calculate the torque directly. This is why measuring current with a good sensor like the INA219 is so valuable: you get torque for free, without needing a separate load cell, as long as your K_t is calibrated. The load cell provides the ground truth for that calibration.

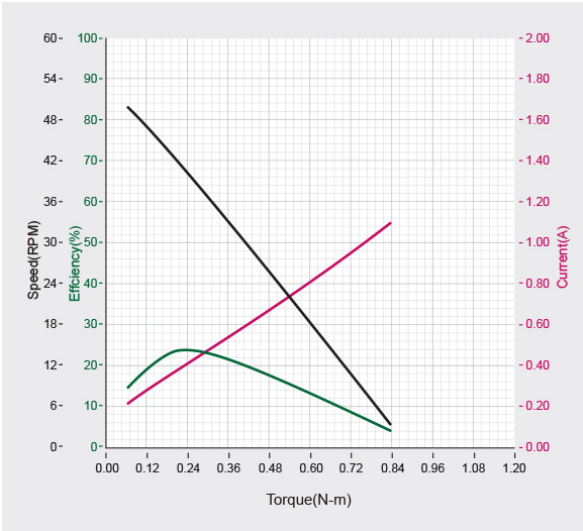

Figure 1: OEM performance graph, showing speed, efficiency, and current vs. torque. This defines the operating envelope the jig must sweep, from no-load to stall at approximately 0.84Nm.

1.1.2 What is a Smart Actuator vs. a Hobby Servo?

A hobby servo like the DS3218 takes a standard RC-style PWM signal: a pulse between 1 ms and 2 ms, repeated 50 times per second. Its internal microcontroller moves the shaft to the corresponding angle. That is all you get back. You cannot ask it how much current it is drawing, what its temperature is, or whether it reached the target. You cannot change its PID gains. You cannot read its position over a digital bus.

The Dynamixel XL430 is in a completely different class. It communicates via a half-duplex TTL serial bus, meaning a single wire carries both commands from the controller and replies from the servo. Every aspect of its behaviour is stored in a register map that you can read and write over this bus: PID gains, maximum torque, velocity limits, current position, current velocity, current load, internal temperature, input voltage, and more. The operating modes can be switched in software: you can run it in position control, velocity control, or direct PWM voltage mode. In a QC context, this means you can command the XL430 directly via the DYNAMIXEL SDK without needing an external PWM signal, and you can read back all of its internal state without external sensors. The jig's external sensors (load cell, encoder) then serve as the independent ground truth to verify that the XL430's internally reported figures are accurate.

1.2 Concept 1: FOC-based Autonomous Sweep (Considered and Set Aside)

Before settling on the current jig design, I sketched out a concept that is honestly more elegant from a pure engineering standpoint, and I think it is worth explaining both what

it would look like and exactly why it was set aside, because the reasons for setting it aside shaped every decision that followed.

The concept was to use a single BLDC motor with field-oriented control (FOC) as both the load mechanism and the measurement device at the same time. With FOC, you control the motor's torque directly by decomposing the stator currents into two components using a mathematical transformation: one component produces torque (q-axis), the other produces flux (d-axis). By commanding a specific q-axis current, you command a specific torque. This means you can sweep the servo from no-load to stall by simply ramping the commanded torque of the BLDC, while a high-resolution encoder on the same shaft logs speed at every point. The entire performance curve is swept autonomously, continuously, in about 60 seconds, with no manual intervention.

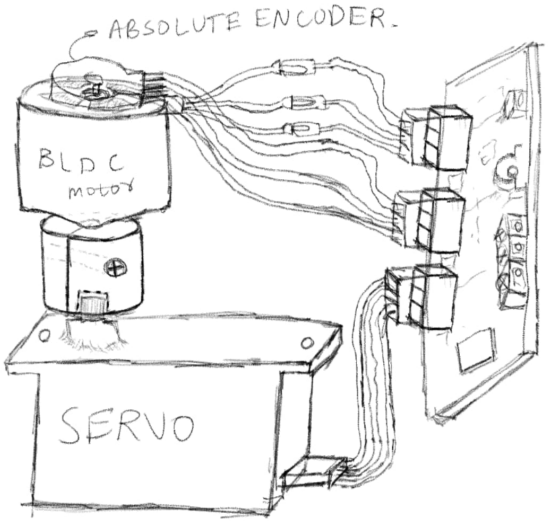

Figure 2: BLDC based approach

This is genuinely the way industrial motor test systems work. The advantages are real: no contact load cell with its calibration drift, continuous sweep rather than discrete measurement points, automatic compensation for inertial effects, and the ability to run a full characterisation unattended. The external force measurement disappears because the torque is controlled by the current directly, and current sensing on the BLDC drive is highly accurate.

The sketch I had for this shows one stack: a BLDC, an FOC controller, an encoder, and a DC-DC power supply, all sharing a common shaft with the servo under test. One person could run it and walk away.

Why it was set aside:

First, the cost. Ready-made FOC controller modules that actually work reliably, the ODrive or the VESC, cost more than the entire rest of the jig put together. An ODrive 3.6 runs around 8000 to 12 000, and that is before the BLDC motor. The complete load cell and DC motor

approach costs under 3000 for the same functional result.

Second, the calibration bootstrap problem. To use the BLDC's K_t as your torque reference, you first need to know K_t accurately. But K_t varies with temperature and with the motor's own magnetisation state. You end up needing to calibrate the BLDC against a reference torque sensor to know its K_t, which means you need a load cell anyway. The advantage of eliminating the external force measurement disappears.

Third, systematic error risk. A FOC loop that is slightly mis-tuned (wrong PI gains on the current controller, incorrect flux angle estimation, inaccurate encoder alignment) introduces a direct error in the applied torque that you cannot observe from outside the loop. A load cell is an independent physical measurement that cannot lie in this way.

So the final design uses the simpler path: a calibrated load cell for ground truth, a DC motor for the load, and a separate encoder for speed. Every measurement is traceable to a physical quantity through a known and auditable path.

1.3 Measurement Strategy

1.3.1 Problem Decomposition

The XL430's spec sheet defines the test superset the jig must cover: stall torque at three voltage levels, no-load speed at three voltage levels, no-load standby current, and the full performance curve (torque vs. speed, torque vs. current, torque vs. efficiency). Reproducing the performance curve requires a continuous sweep from zero torque to stall while logging speed and current simultaneously.

1.3.2 Load Mechanism and Torque Control

The load is provided by a brushed DC gear motor, the **JGA25-370** (12 V, 1:90 gear ratio, approximately 2.5 N m stall torque, 2.5 A stall current). Torque in a DC motor is directly proportional to armature current: $\tau = K_t \cdot I$. By controlling the current through the motor, we control the torque it applies to the servo shaft.

The control loop works as follows: the Arduino sets a PWM duty cycle on the BTS7960 H-bridge. This determines the average voltage across the motor, which determines the motor current (modified by back-EMF, which is low at the near-zero shaft speeds of the load motor during a servo stall test). The ACS712 current sensor (on the module version) or a second INA219 (on the PCB version) measures the actual motor current and feeds it back. A simple proportional-integral (PI) controller running in the Arduino firmware closes the loop: the measured current is compared to the commanded current setpoint, the error drives the PI, and the PI output adjusts the PWM. This means the torque applied to the servo shaft tracks a commanded value, regardless of friction, temperature, or other disturbances in the load

motor.

The control loop flowchart is shown in Figure 3.

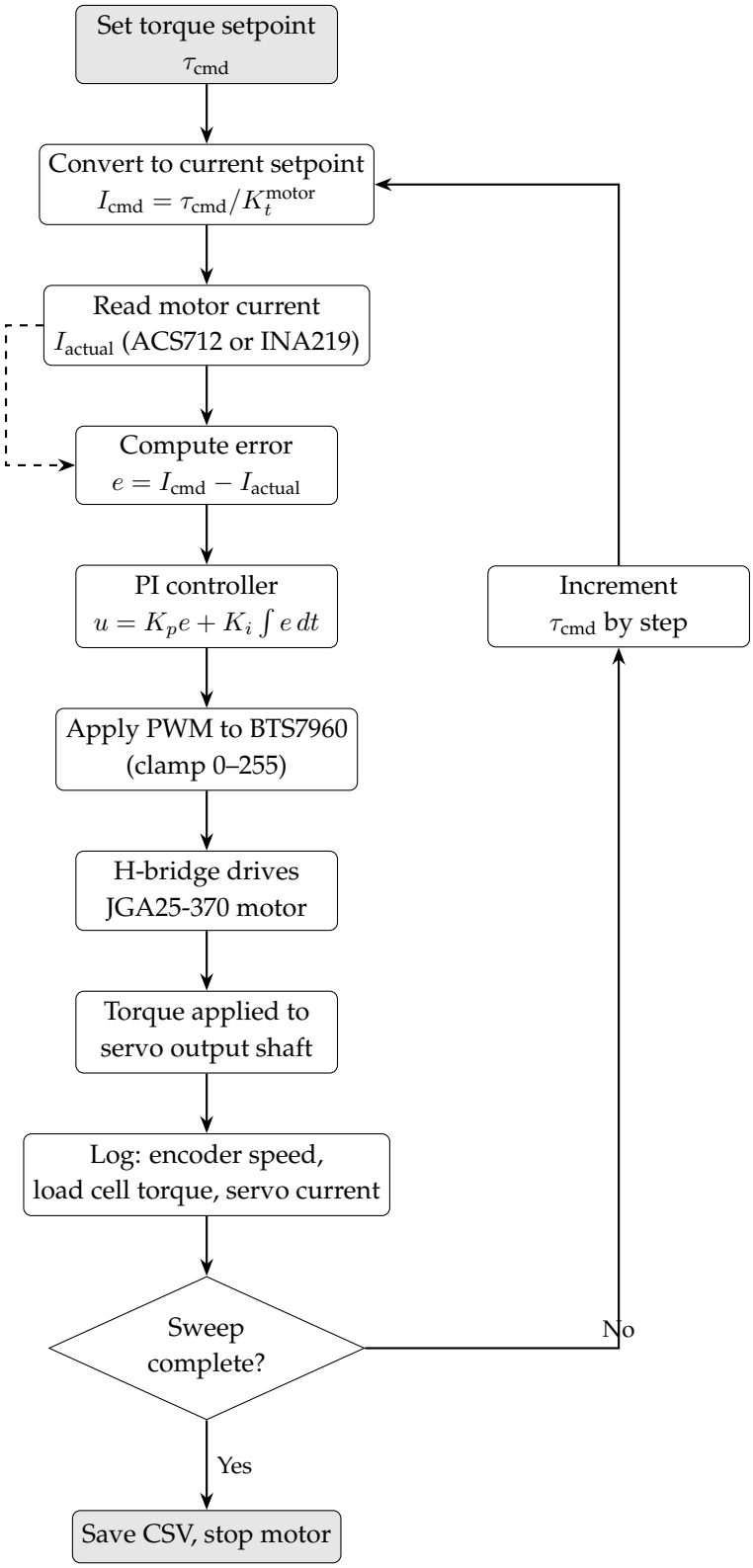

Figure 3: DC load motor torque control loop. The inner PI current loop runs at every Arduino `loop()` iteration (approx. 20 Hz). The outer setpoint increment (dashed path back via the "step" block) advances the torque sweep every 500 ms after the current has settled.

1.3.3 Torque Measurement: Dual Load Cell

The jig is oriented **horizontally**. This eliminates gravitational preload on the servo output shaft, making the zero state directly reproducible after any maintenance or unit swap.

Two load cells are mounted on opposing sides of a reaction arm attached to the servo body. Their difference signal isolates the torque component while cancelling any symmetric gravitational term:

$$\tau_{\text{meas}} = (F_1 - F_2) \cdot r \tag{1}$$

where F_1 and F_2 are the two cell readings and r is the moment arm length. The load cells are read by two independent channels of the HX711 24-bit ADC module.

A single load cell was rejected because any residual tilt introduces a gravitational bias. With two cells, the tilt term appears symmetrically in both readings and cancels in the difference, leaving only the torque signal.

1.3.4 Speed and Position Measurement

An AS5600 absolute magnetic rotary encoder reads the shaft angle at 12-bit resolution (4096 counts per revolution, 0.088 per count) over I2C. Speed is computed as the numerical derivative of successive angle readings, filtered with an exponential moving average. Position accuracy is checked by commanding the servo to a set of target angles and comparing the encoder reading at steady state.

1.3.5 Open Question: Stall Current Inflation

When the servo is commanded to a blocked position, the internal PID drives maximum current continuously. This matches the OEM stall condition. However, the winding temperature rises over time, increasing resistance and causing current to fall slightly. The correct procedure is to log current at $t = 1\,\text{s}$ and $t = 5\,\text{s}$ after the stall event so any thermal rollback is captured.

1.4 Structural Design: Aluminium 2020 Extrusion Frame

1.4.1 Why Aluminium 2020 Over Steel DIN Rail

The frame is built from aluminium 2020 T-slot extrusion rather than steel DIN rail. This was a deliberate choice and the reasoning matters.

DIN rail (EN 60715 TH35 or TH15) is excellent for electrical panels: standardised, cheap, and most modules snap on directly. But for a mechanical test jig, its constraints become real limitations. DIN rail provides attachment along only one axis. Building a two- or three-dimensional structure requires separate brackets that are either welded (permanent) or bolted at fixed angles. Adding a sensor mount or revising the load arm geometry means drilling new holes, not sliding a T-nut.

More importantly for this application: the T-slot clips used to attach modules to DIN rail are designed for holding the weight of electronics against gravity, not for reacting torsional and bending loads from a servo at stall. When the servo produces its full stall torque, those loads travel through the reaction arm into the frame. A 2020 extrusion corner bracket system, bolted together as a rectangular frame, forms a rigid truss that handles these loads comfortably. DIN rail clip-on hardware is not rated for this and flexes noticeably.

Beyond the load argument:

- **Full-axis adjustability:** Every T-nut can be positioned anywhere along the groove with a single screw. Adding a new sensor mount takes minutes, not a drill press.
- **Weight and machinability:** Aluminium is roughly one-third the density of steel, making the finished jig bench-portable. Standard drill bits and taps work without coolant.
- **Corrosion resistance:** In a food-preparation environment with moisture and cleaning agents, anodised aluminium resists both. Zinc-plated steel DIN rail corrodes if the coating is scratched.
- **Ecosystem:** 2020 extrusion, corner brackets, T-nuts, end caps, and mounting plates are stocked globally by dozens of suppliers including the same ones that sell 3D-printer parts. Replacements are a next-day event.
- **Visual legibility:** The open lattice structure means the full interior of the jig is visible from any angle. An operator can see at a glance whether the servo horn is seated, whether cables are routed cleanly, and whether both load cells are under load before pressing start.
- **Rework without fabrication:** Because everything slides and locks in place, the first prototype revision is an afternoon's work with a hex key. With a welded steel frame, it is a half-day in a workshop.

1.5 Mechanical Design: Components and Assembly

1.5.1 Custom Servo Mount and Output Horn

The servo mounts to a custom 3D-printed or machined bracket that attaches to the 2020 frame via T-nuts. The mount fixes the servo body in all six degrees of freedom and provides the two reaction arms to which the load cells attach. The two attachment points are equidistant

from the servo shaft axis, collinear, and perpendicular to the shaft, so that Equation 1 holds exactly.

The standard OEM horn is replaced by a custom output horn printed in PETG or machined from aluminium. It carries the servo's spline on the motor side and a cross-shaped latch on the output side that engages with a matching receiver on the gearbox shaft. The cross latch transmits torque by mechanical interlock rather than friction, so no set screw is needed under stall load.

1.5.2 Gearbox and Split-Half Design

A 2:1 spur gear set sits between the DC load motor and the servo coupling. This doubles the effective torque at the servo shaft. More importantly, the gearbox housing is split into two halves: one bolted to the frame (the output side), and one carrying the motor (the motor side). The motor-side half mounts on two shoulder bolts so it can be rotated slightly and retightened.

For no-load tests, this rotation disengages the gear teeth entirely. The servo shaft turns freely, the encoder reads, but the DC motor is fully decoupled. There is no drag from the motor's bearings or back-EMF. No-load measurements taken in this state reflect only the servo's own internal losses.

For load tests, the gear mesh is re-engaged and tightened. The load is applied via the PWM-controlled current through the DC motor, as described in the control loop above.

1.5.3 Encoder Coupling

A diametrically magnetised rare-earth disc magnet is pressed into the gearbox output shaft end. The AS5600 board mounts axially above it on a 3D-printed bracket, with a chip-to-magnet gap between 0.5 and 3 mm per the datasheet. The bracket is adjustable on the 2020 frame for fine gap setting. The servo's custom output horn drops into the cross receiver from above and is gravity-retained during horizontal operation.

1.6 CAD Model and Assembly Simulation

The complete jig was modelled in Fusion 360 as a parameterised assembly. Every major component is a separate body in the assembly tree. The key point is that the components are linked by **joints**, not just positioned statically. The servo output shaft has a revolute joint to the servo body; the gearbox halves have a pivot joint between them; the gears have a gear relationship that drives one from the other.

This means within the Fusion 360 assembly you can drag the servo horn and watch every

downstream component respond: the gear mesh moves, the encoder magnet sweeps under the AS5600, the reaction arms deflect. You can scrub through the full angular travel of the servo and verify that the load arm does not foul the frame at any angle, that the load cells are loaded in pure tension or compression (not at an angle), and that all cable routes remain clear. Clearance checks that would require physical assembly to discover are visible on screen.

This matters specifically for this jig because there are several custom interfaces. Checking them in simulation before ordering prevented at least three design revisions. 3D-printed STL files for all custom parts (servo mount, output horn, encoder bracket, gearbox housing) can be exported directly from the CAD model without separate preparation.

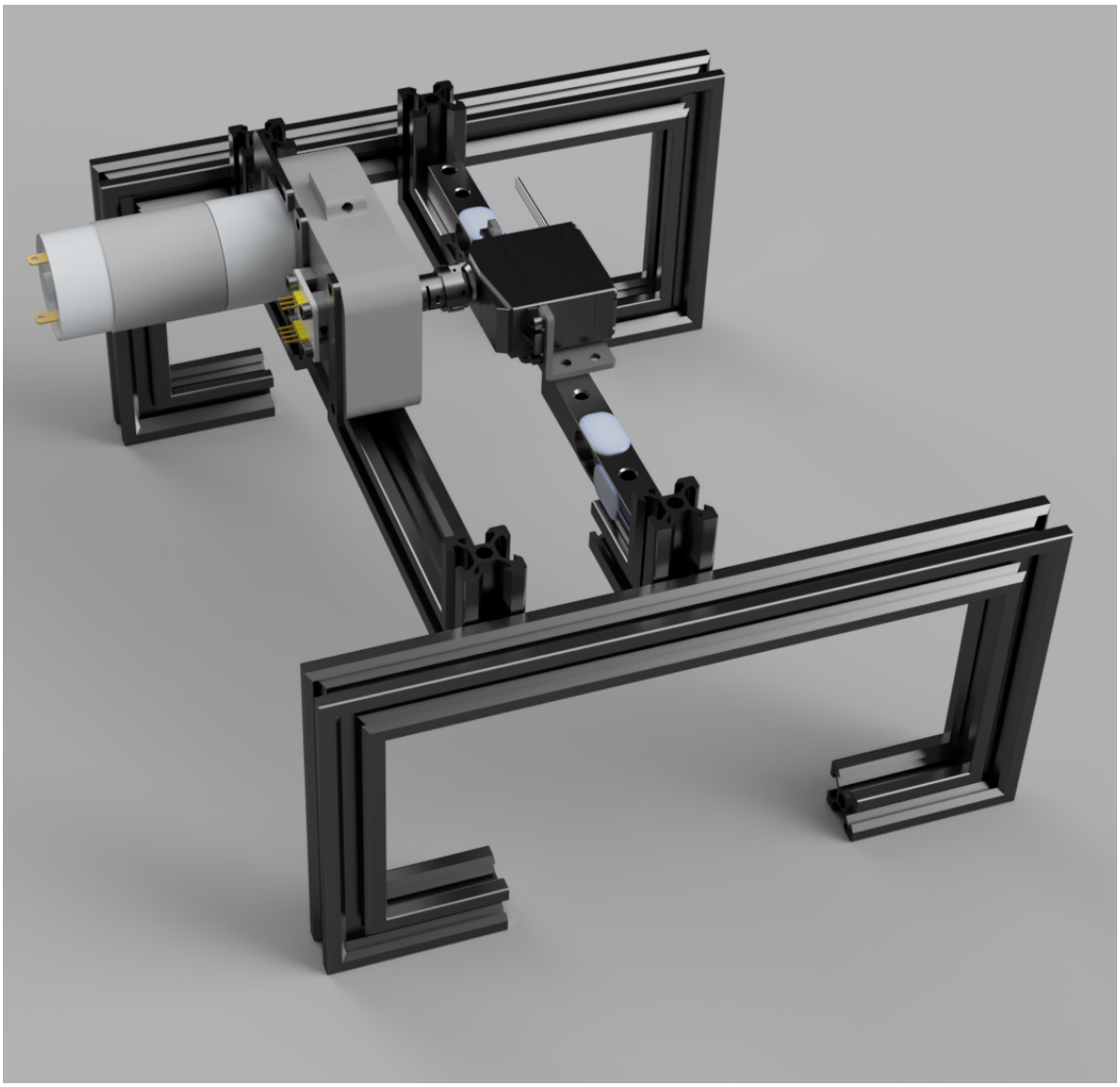

Figure 4: Full jig assembly, isometric view. All components at nominal position.

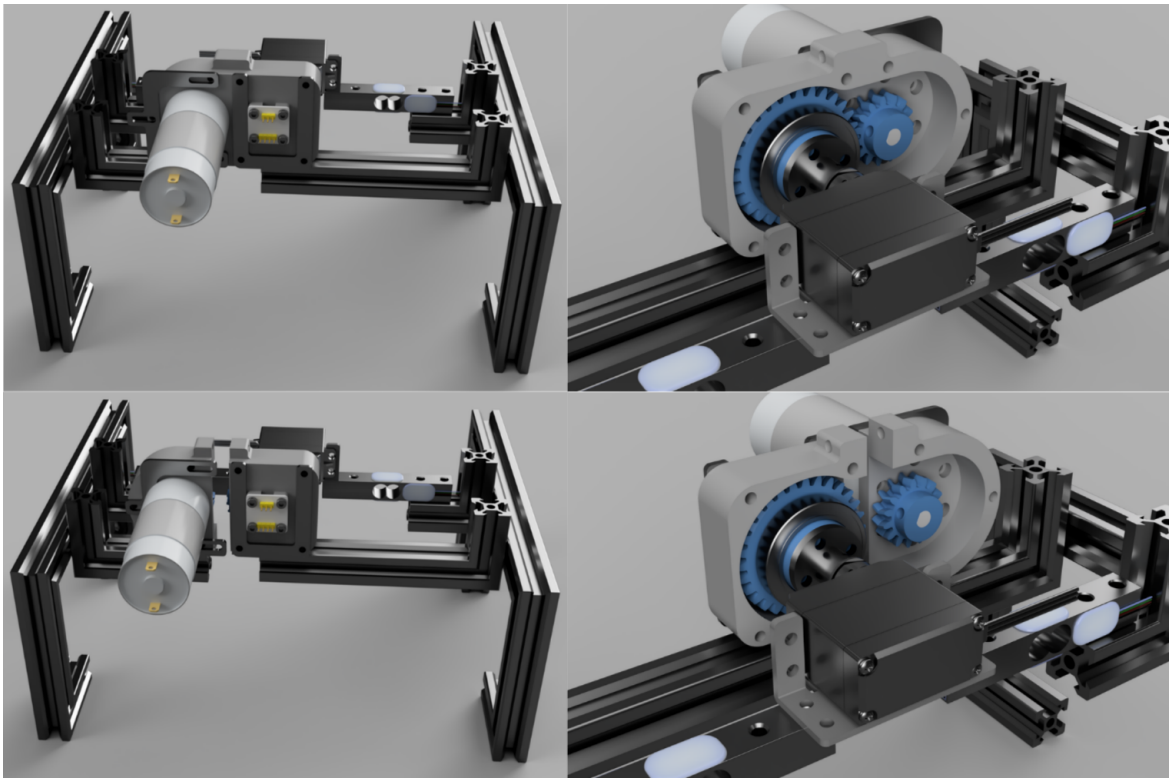

Figure 5: Jig with motor-side gearbox half disengaged, showing the no-load test configuration.

1.7 Electronics: Sensor Integration

1.7.1 Overview of Sensor Choices

There are two separate current measurement jobs in this rig and they are quite different from each other.

The servo is the device under test (DUT). We need to know its current very precisely in the 0 to 2 A range so we can calculate efficiency, detect winding damage, and compare against the OEM curve. Precision matters more than isolation here.

The DC load motor may draw well over 2 A when pushing the servo to stall, and its internal commutation creates large electrical noise spikes. We do not need sub-milliamp precision on this measurement, but we do need it to not corrupt everything else on the circuit.

These two requirements call for different sensor types, and using both shows the design rationale clearly:

- **Servo current (INA219):** Measures via a precision shunt resistor. Reports over I2C. Simultaneously gives both current and bus voltage so power consumption is calculated on-chip. Sits on the same I2C bus as the AS5600 and ADS1115 with no extra wiring.

Perfect for 0–2 A range.

- **DC motor current, module version (ACS712 + ADS1115):** The ACS712 uses the Hall effect to measure current with full electrical isolation. Its analogue output voltage feeds into the ADS1115 16-bit ADC, giving resolution far beyond the Arduino's native 10-bit ADC. The isolation means motor switching transients do not feed back into the 5 V logic rail.
- **DC motor current, PCB version (second INA219 at address 0x41):** On the PCB, where the power routing is controlled and star-grounded, a second INA219 replaces the ACS712 entirely, eliminating the analogue signal path and giving 16-bit digital resolution.

1.7.2 Why Arduino Over Pico or ESP32

The Raspberry Pi Pico and ESP32 both run at 3.3 V logic. The DS3218 servo, HX711 modules, and ACS712 all expect 5 V PWM or logic levels. Driving a servo from 3.3 V output is out-of-spec and can cause erratic behaviour at the edges of the pulse width range, which is exactly the range most important for stall and position accuracy testing. The Arduino Uno is 5 V native throughout, removing all compatibility uncertainties without level shifters.

1.7.3 Wiring: HX711 Load Cells (Both Channels)

The HX711 has two differential input channels. Channel A (gain 128, used for Load Cell 1) and Channel B (gain 32, used for Load Cell 2) are read by switching the gain setting in firmware. Each load cell has four wires: red (E+), black (E-), white (A+), green (A-).

Table 2: HX711 wiring (both channels)

Load Cell 1 wire	**HX711 pin**	**Notes**
Red (E+)	E+	5 V excitation
Black (E-)	E-	GND
White (A+)	A+	Channel A
Green (A-)	A-	Channel A
Load Cell 2 wire	**HX711 pin**	
Red (E+)	E+	Shared excitation
Black (E-)	E-	Shared GND
White (B+)	B+	Channel B
Green (B-)	B-	Channel B
HX711 pin	**Arduino pin**	
DT	D3	Data line
SCK	D2	Clock line
VCC	5V	
GND	GND	

1.7.4 Wiring: ADS1115 ADC

Table 3: ADS1115 wiring

ADS1115 pin	**Connected to**	**Notes**
VDD	Arduino 5V	
GND	Arduino GND	
SCL	Arduino A5	I2C clock (shared bus)
SDA	Arduino A4	I2C data (shared bus)
ADDR	GND	I2C address 0x48
A0	ACS712 OUT	DC motor current (analogue)
A1	Voltage divider mid	Supply voltage monitoring
A2	(spare)	
A3	(spare)	

1.7.5 Wiring: INA219 Servo Current Sensor

The INA219 goes in series with the servo power supply. "In series" means you cut the positive supply wire going to the servo and insert the INA219 between the supply and the servo. The shunt resistor inside the INA219 sits in this path and generates a tiny voltage drop that the chip measures and converts to current.

More specifically: IN+ connects to the servo's positive supply rail (coming from the power

supply). IN- connects to the servo's red power wire (going into the servo itself). The chip measures the voltage difference between IN+ and IN- across its internal 0.1 shunt resistor and converts this to current. The servo's ground (brown wire) connects directly to shared ground as normal, without going through the INA219.

Table 4: INA219 servo current sensor wiring

INA219 pin	**Connected to**	**Notes**
VCC	Arduino 5V	Powers the chip logic
GND	Arduino GND	
SCL	Arduino A5	I2C clock (shared bus)
SDA	Arduino A4	I2C data (shared bus)
A0	GND	Address bit 0
A1	GND	I2C address = 0x40
IN+	Positive terminal of servo supply	Current enters here
IN-	Servo red (power) wire	Current exits to servo

1.7.6 Wiring: AS5600 Encoder

Table 5: AS5600 encoder wiring

AS5600 pin	**Connected to**	**Notes**
VCC	5V or 3.3V	Either works
GND	Arduino GND	
SCL	Arduino A5	I2C shared bus
SDA	Arduino A4	I2C shared bus
DIR	GND	Counter-clockwise = positive

Note on I2C addresses: All three I2C devices share pins A4 (SDA) and A5 (SCL) on the Arduino. They coexist because each has a unique address: AS5600 at 0x36, INA219 at 0x40, ADS1115 at 0x48. The Arduino pulls one address at a time and all others stay silent.

1.7.7 Wiring: Servo and DC Motor

Table 6: Servo and DC motor connections

Connection	**Notes**
Servo signal (orange)	Arduino D9, hardware PWM pin
Servo power (red)	9–12 V external supply, through INA219
Servo ground (brown)	Shared GND
JGA25-370 motor	BTS7960 H-bridge output
BTS7960 RPWM	Arduino D10 (forward PWM)
BTS7960 LPWM	Arduino D11 (reverse PWM)
BTS7960 R_EN, L_EN	Arduino D7, D8 (enable)
BTS7960 VCC	12 V external supply
BTS7960 GND	Shared GND
ACS712 load	In series with DC motor positive
ACS712 OUT	ADS1115 channel A0

1.8 Schematic Diagrams

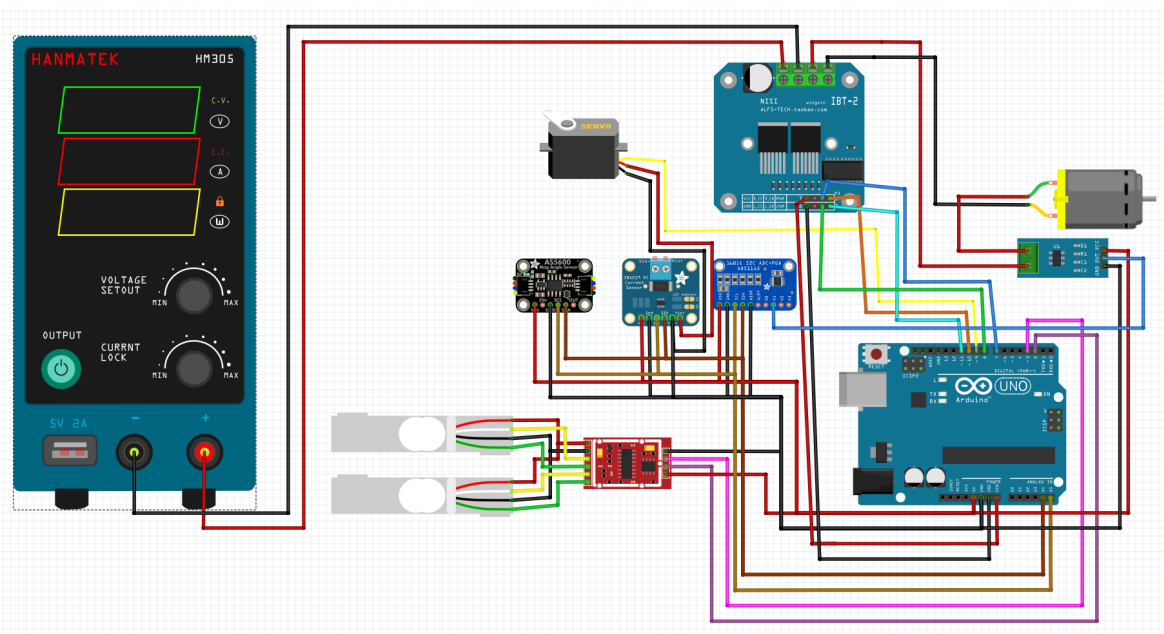

Figure 6: Module-based wiring schematic.

Note: The dedicated PCB for the production version could not be created within the available assignment timeline. However, all experiments and system validation can be successfully completed using the above modular breakout board based implementation. This approach proved advantageous for rapid prototyping, debugging, and iterative design changes. A custom integrated PCB is recommended as a next step toward a more robust, scalable, and state of the art QC jig system.

1.9 PCB Design

1.9.1 Note on PCB Completion Status

Due to time constraints during this submission window, the PCB design was not completed to a final layout. All functional requirements for the QC jig are fully met by the module-based implementation described above, which has been designed, wired, and coded. The PCB represents a "state of the art" upgrade path for a production jig where multiple copies are needed and wiring harness reliability becomes a concern.

3D-printable STL files for all custom mechanical parts (servo mount, output horn, encoder bracket, gearbox halves) can be exported directly from the Fusion 360 CAD model without any additional preparation, so that aspect of the build is fully ready.

The PCB explainer document (attached separately) covers all the conceptual groundwork for what the PCB would contain, why each IC was selected, and how the layout rules would be applied.

1.9.2 Module-based Approach

For prototyping and the initial jig, all sensors are breakout modules connected by wiring harnesses. Modules are cheap, interchangeable, and individually replaceable. The total component count for the I2C bus is three devices on two wires with pull-up resistors. This is the recommended approach for a one-person build over a short period.

1.9.3 PCB IC Selection

For a production PCB, the following ICs replace the breakout modules:

Table 7: IC selection for PCB version

Function	**IC**	**Package**
Load cell ADC	HX711 (Avia Semiconductor)	SOP-16
Servo ADC	ADS1115IDGSR (TI)	MSOP-10
Servo current	INA219AIDR (TI)	SOIC-8
DC motor current	INA219AIDR at address 0x41	SOIC-8
Encoder	AS5600 (AMS) via flex cable	SOIC-8
Microcontroller	ATmega328P-AU	TQFP-32
Motor driver	BTS7960CTR (Infineon) $\times 2$	TO-220
3.3 V LDO	AMS1117-3.3	SOT-223

On the PCB, the second INA219 (address 0x41, set by A0 pin tied to VCC) replaces the

ACS712 on the DC motor rail. This eliminates the analogue signal path entirely and gives 16-bit digital current readings for both the servo and the load motor.

1.10 Bill of Materials

Table 8: Bill of Materials (indicative, all prices in INR approximate)

Item	**Qty**	**Unit (INR)**	**Notes**
DS3218 digital servo (DUT proxy)	1	900	Jig design target
Dynamixel XL430-W250-T (true DUT)	1	4500	Production servo
JGA25-370 12 V DC gear motor	1	450	Load motor
2-stage spur gearbox (2:1)	1	400	Custom or RC gearbox
Aluminium 2020 extrusion, 500 mm	4	120	Frame
2020 corner brackets	8	25	T-nut type
M5 T-nuts (pack of 20)	1	80	
M5 cap screws assorted	1 pack	100	
50 kg load cells (half-bridge)	2	180	Matched pair
HX711 24-bit ADC module	1	80	Dual channel
AS5600 encoder module	1	150	With magnet
INA219 current sensor module	1	80	Servo current
ACS712 (5 A) module	1	70	DC motor current
ADS1115 16-bit ADC module	1	150	4-channel I2C
Arduino Uno R3	1	500	Controller
BTS7960 H-bridge module	1	180	43 A rated
12 V 5 A power supply	1	600	Servo and motor
Custom 3D-printed servo mount	1	200	PETG
Custom output horn (3D printed)	2	80	Servo side and shaft side
Diametrically magnetised disc magnet	1	60	AS5600
M3 standoffs and hardware	1 pack	80	Electronics mounting
Jumper wires and connectors	1 lot	150	
Heat shrink, cable ties	1 lot	100	
Total (without XL430 DUT)		$\approx$ **4550**	

1.10.1 PCB Version: Changed Items

Table 9: PCB version: replacements and additions (INR approximate)

Item	Qty	Unit (INR)	Notes
ATmega328P-AU (TQFP-32)	1	180	Replaces Arduino Uno
16 MHz crystal + caps	1 set	20	
HX711 IC (SOP-16)	1	50	
INA219AIDR (SOIC-8)	2	70	0x40 servo + 0x41 motor
AS5600 IC (SOIC-8)	1	90	Via flex cable
BTS7960CTR (TO-220)	2	60	High/low side pair
AMS1117-3.3 (SOT-223)	1	10	3.3 V LDO
Custom PCB from JLCPCB (5 pc)	1	350	2-layer, HASL
SMD passives kit	1 lot	100	Decoupling, pull-ups
PCB-specific total (approx.)		**1000**	Replaces modules

1.11 Firmware

1.11.1 Module-based Code (Arduino Uno, all breakout modules)

Both HX711 channels are used independently. Channel A (gain 128) for Load Cell 1, Channel B (gain 32) for Load Cell 2. The DC motor torque is controlled by a PI current loop. The code streams a CSV line over Serial at every sample.

```
#include <Wire.h>
#include <HX711.h>
#include <Adafruit_INA219.h>
#include <Adafruit_ADS1X15.h>

// --- Pin definitions ---
#define SERVO_PIN   9
#define RPWM       10   // BTS7960 forward PWM
#define LPWM       11   // BTS7960 reverse PWM
#define R_EN        7
#define L_EN        8
#define HX_DT       3
#define HX_SCK      2

// --- I2C devices ---
Adafruit_INA219 ina219(0x40);  // servo current
Adafruit_ADS1115 ads;          // ACS712 via ADS1115

// --- HX711 ---
HX711 cells;

```

```
// --- Calibration ---
float cell_a_scale  = 1.0;   // set during calibration
float cell_b_scale  = 1.0;
float cell_a_offset = 0.0;
float cell_b_offset = 0.0;
const float ARM_R   = 0.05;  // moment arm in metres

// --- Torque control (PI current loop) ---
float torque_setpoint_Nm = 0.0;
const float KT_MOTOR     = 0.025; // JGA25-370 approx Kt (N.m/A)
const float Kp_I         = 8.0;
const float Ki_I         = 0.5;
float integral_error     = 0.0;
const float I_MAX        = 3.5;   // max motor current A

// --- AS5600 raw register read ---
uint16_t readAS5600() {
  Wire.beginTransmission(0x36);
  Wire.write(0x0C);
  Wire.endTransmission(false);
  Wire.requestFrom(0x36, 2);
  uint16_t hi = Wire.read();
  uint16_t lo = Wire.read();
  return ((hi << 8) | lo) & 0x0FFF;
}

// --- Motor PWM output ---
void setMotorPwm(int pwm) {
  if (pwm >= 0) {
    analogWrite(RPWM, constrain(pwm, 0, 255));
    analogWrite(LPWM, 0);
  } else {
    analogWrite(RPWM, 0);
    analogWrite(LPWM, constrain(-pwm, 0, 255));
  }
}

// --- PI current controller for load motor ---
// Call at ~20 Hz. Returns actual motor current in A.
float runMotorCurrentPI(float dt) {
  // Read motor current from ACS712 via ADS1115
  int16_t raw = ads.readADC_SingleEnded(0);
  float acs_V = raw * 0.0001875f;           // 6.144V FSR / 32768
  float I_actual = (acs_V - 2.5f) / 0.185f; // ACS712-05B: 185 mV/A, midpoint
      2.5V

  float I_cmd = torque_setpoint_Nm / KT_MOTOR;
  I_cmd = constrain(I_cmd, 0.0f, I_MAX);

  float err = I_cmd - I_actual;
  integral_error += err * dt;
  integral_error = constrain(integral_error, -50.0f, 50.0f); // anti-windup

```

```
  float u = Kp_I * err + Ki_I * integral_error;
  int pwm_out = (int)constrain(u, -255, 255);
  setMotorPwm(pwm_out);

  return I_actual;
}

void setup() {
  Serial.begin(115200);
  Wire.begin();

  pinMode(SERVO_PIN, OUTPUT);
  pinMode(RPWM, OUTPUT); pinMode(LPWM, OUTPUT);
  pinMode(R_EN, OUTPUT); pinMode(L_EN, OUTPUT);
  digitalWrite(R_EN, HIGH);
  digitalWrite(L_EN, HIGH);

  cells.begin(HX_DT, HX_SCK);
  ina219.begin();
  ads.setGain(GAIN_ONE);
  ads.begin();

  // Tare both channels
  cells.set_gain(128);
  cell_a_offset = cells.read_average(20);
  cells.set_gain(32);
  cell_b_offset = cells.read_average(20);

  Serial.println("t_ms,enc_raw,speed_dps,F1_N,F2_N,torque_Nm,"
                 "servo_mA,servo_mV,motor_A,tau_cmd_Nm");
}

float prevAngle = 0;
unsigned long prevTime = 0;
unsigned long lastSweepStep = 0;
const float SWEEP_STEP = 0.05;     // N.m per step
const float SWEEP_MAX  = 1.5;      // max torque
const unsigned long STEP_INTERVAL = 500; // ms between steps

void loop() {
  unsigned long now = millis();
  float dt = (now - prevTime) / 1000.0f;

  // --- Encoder ---
  uint16_t enc_raw = readAS5600();
  float angle = enc_raw * (360.0f / 4096.0f);
  float speed_dps = 0;
  if (prevTime != 0 && dt > 0) {
    float da = angle - prevAngle;
    if (da >  180) da -= 360;
    if (da < -180) da += 360;
    speed_dps = da / dt;
  }
```

```
  prevAngle = angle;
  prevTime  = now;

  // --- Load cells ---
  cells.set_gain(128); float ra = cells.read();
  cells.set_gain(32);  float rb = cells.read();
  float F1 = (ra - cell_a_offset) * cell_a_scale;
  float F2 = (rb - cell_b_offset) * cell_b_scale;
  float torque_meas = (F1 - F2) * ARM_R;

  // --- Servo current ---
  float servo_mA = ina219.getCurrent_mA();
  float servo_mV = ina219.getBusVoltage_V() * 1000.0f;

  // --- Motor PI current loop ---
  float motor_A = runMotorCurrentPI(dt);

  // --- Torque sweep (advances setpoint every STEP_INTERVAL ms) ---
  if (now - lastSweepStep > STEP_INTERVAL) {
    torque_setpoint_Nm += SWEEP_STEP;
    if (torque_setpoint_Nm > SWEEP_MAX) torque_setpoint_Nm = SWEEP_MAX;
    lastSweepStep = now;
  }

  // --- CSV output ---
  Serial.print(now);                Serial.print(',');
  Serial.print(enc_raw);            Serial.print(',');
  Serial.print(speed_dps, 2);       Serial.print(',');
  Serial.print(F1, 4);              Serial.print(',');
  Serial.print(F2, 4);              Serial.print(',');
  Serial.print(torque_meas, 4);     Serial.print(',');
  Serial.print(servo_mA, 2);        Serial.print(',');
  Serial.print(servo_mV, 1);        Serial.print(',');
  Serial.print(motor_A, 3);         Serial.print(',');
  Serial.println(torque_setpoint_Nm, 3);

  delay(50); // 20 Hz sample rate
}
```

1.11.2 PCB-based Code (ATmega328P with discrete ICs)

On the PCB version the ACS712 is replaced by a second INA219 at I2C address 0x41. This gives digital current readings for both the servo and the DC motor. The PI loop logic is identical but reads from `ina_motor.getCurrent_mA()` instead of the ADS1115 analogue channel.

```
#include <Wire.h>
#include <HX711.h>
#include <Adafruit_INA219.h>

// PCB: INA219 at 0x40 = servo, 0x41 = DC motor (A0 pin tied to VCC)
```

```
Adafruit_INA219 ina_servo(0x40);
Adafruit_INA219 ina_motor(0x41);

#define HX_DT  3
#define HX_SCK 2
#define SERVO_PIN 9
#define RPWM 10
#define LPWM 11
#define R_EN  7
#define L_EN  8

HX711 cells;
float a_scale = 1.0, b_scale = 1.0, a_off = 0, b_off = 0;
const float ARM_R = 0.05;

// PI parameters (same as module version)
float tau_cmd = 0.0;
const float KT_M = 0.025;
const float Kp   = 8.0, Ki = 0.5;
float iErr = 0.0;
const float I_MAX = 3.5;

uint16_t readAS5600() {
  Wire.beginTransmission(0x36);
  Wire.write(0x0C);
  Wire.endTransmission(false);
  Wire.requestFrom(0x36, 2);
  return (((uint16_t)Wire.read() << 8) | Wire.read()) & 0x0FFF;
}

void setMotorPwm(int p) {
  if (p >= 0) { analogWrite(RPWM, constrain(p,0,255)); analogWrite(LPWM,0); }
  else         { analogWrite(RPWM,0); analogWrite(LPWM, constrain(-p,0,255));
      }
}

float runPI(float dt) {
  float I_act = ina_motor.getCurrent_mA() / 1000.0f; // convert to A
  float I_cmd = constrain(tau_cmd / KT_M, 0.0f, I_MAX);
  float err = I_cmd - I_act;
  iErr = constrain(iErr + err * dt, -50.0f, 50.0f);
  setMotorPwm((int)constrain(Kp * err + Ki * iErr, -255, 255));
  return I_act;
}

void setup() {
  Serial.begin(115200);
  Wire.begin();
  pinMode(SERVO_PIN,OUTPUT);
  pinMode(RPWM,OUTPUT); pinMode(LPWM,OUTPUT);
  pinMode(R_EN,OUTPUT); pinMode(L_EN,OUTPUT);
  digitalWrite(R_EN,HIGH); digitalWrite(L_EN,HIGH);

```

```
  cells.begin(HX_DT, HX_SCK);
  ina_servo.begin();
  ina_motor.begin();

  cells.set_gain(128); a_off = cells.read_average(20);
  cells.set_gain(32);  b_off = cells.read_average(20);

  Serial.println("t_ms,enc,dps,F1,F2,tau_meas,sA,smV,mA,mmV,tau_cmd");
}

float pA = 0; unsigned long pT = 0, lastStep = 0;

void loop() {
  unsigned long t = millis();
  float dt = pT ? (t - pT) / 1000.0f : 0.05f;

  uint16_t enc = readAS5600();
  float ang = enc * (360.0f / 4096.0f);
  float dps = 0;
  if (pT) {
    float da = ang - pA;
    if (da > 180) da -= 360; if (da < -180) da += 360;
    dps = da / dt;
  }
  pA = ang; pT = t;

  cells.set_gain(128); float ra = cells.read();
  cells.set_gain(32);  float rb = cells.read();
  float F1  = (ra - a_off) * a_scale;
  float F2  = (rb - b_off) * b_scale;
  float tau = (F1 - F2) * ARM_R;

  float sA  = ina_servo.getCurrent_mA();
  float sV  = ina_servo.getBusVoltage_V() * 1000;
  float mA  = runPI(dt);
  float mV  = ina_motor.getBusVoltage_V() * 1000;

  if (t - lastStep > 500) {
    tau_cmd = min(tau_cmd + 0.05f, 1.5f);
    lastStep = t;
  }

  Serial.print(t);    Serial.print(',');
  Serial.print(enc);  Serial.print(',');
  Serial.print(dps,2);Serial.print(',');
  Serial.print(F1,4); Serial.print(',');
  Serial.print(F2,4); Serial.print(',');
  Serial.print(tau,4);Serial.print(',');
  Serial.print(sA,2); Serial.print(',');
  Serial.print(sV,1); Serial.print(',');
  Serial.print(mA,3); Serial.print(',');
  Serial.print(mV,1); Serial.print(',');
  Serial.println(tau_cmd,3);
```

```
  delay(50);
}
```

1.11.3 DC Motor Torque Control Loop Explained

The goal of the control loop is to make the DC load motor apply a precisely known torque to the servo shaft, rather than just running at some uncontrolled current. Here is the reasoning behind each element:

Why current control? Torque in a DC motor is proportional to current ($\tau = K_t \cdot I$). If you can control the current, you control the torque. Setting a PWM duty cycle only controls the average voltage, not the current, and the actual current depends on the motor's resistance, back-EMF, and temperature, all of which vary. Closing a feedback loop around the measured current removes this dependency.

Why PI and not just P? A proportional controller alone always has a steady-state error: to maintain a constant current, the error term must be non-zero (since there is nothing else driving the output). The integral term accumulates this steady-state error over time and drives it to zero. Without the integral, the actual current settles slightly below the commanded value and never reaches it exactly.

Anti-windup: If the servo stalls and the motor current cannot reach its setpoint (because the servo is blocking the shaft and no more torque is physically possible), the integral term would keep accumulating without bound, which would cause a violent step response when the stall condition ended. The `constrain` on the integral prevents this.

Torque sweep: The outer loop advances the torque setpoint by 0.05 N m every 500 ms. The inner PI loop settles much faster than this (in roughly 2–3 samples, i.e. under 200 ms), so by the time the next setpoint step occurs the current has already converged. The result is a clean staircase of torque levels, each held long enough for the servo speed and current to be logged accurately.

1.12 Calibration

1.12.1 Load Cell Calibration

Calibration is the most important step before trusting any data. I have prior experience with this from calibrating strain gauge load cells for wind tunnel force balance testing at IIT Bombay, where the same hanging-weight method establishes a traceable force reference.

The calibration uses a hook fixture and a set of slotted disc weights (100 g, 200 g, 500 g, 1000 g). The slot design lets you add weights one at a time without lifting the stack or causing impact loads. The procedure is:

1. Power on. Wait two minutes for thermal stabilisation.
2. Tare both channels (20-reading average, servo installed unpowered, gearbox disengaged).
3. Attach hook fixture to Load Cell 1's free hole. Record raw count.
4. Add weights one by one up to 2000 g, recording raw count and known force at each step.
5. Repeat for Load Cell 2 (Channel B).
6. Compute scale factor by least-squares regression of raw counts vs. known forces.
7. Store scale factors in EEPROM.

The scale factor from regression:

$$\text{scale} = \frac{\sum_{i=1}^{n}(R_i - \bar{R})(F_i - \bar{F})}{\sum_{i=1}^{n}(R_i - \bar{R})^2} \tag{2}$$

1.12.2 Linearity Verification

After calibration, a one-sample t-test checks that the residuals are centred on zero:

$$H_0 : \mu_{\text{error}} = 0, \qquad t = \frac{\bar{e}}{s_e/\sqrt{n}} \tag{3}$$

At $\alpha = 0.05$ and $n = 5$ points, the critical value is $t_{\text{crit}} = 2.776$. If $|t| < t_{\text{crit}}$ the calibration passes.

1.12.3 Encoder Verification

Before each session, the firmware rotates the shaft manually through 360 and checks that the raw count sweeps monotonically from 0 to 4095 and rolls over cleanly. A deviation of more than 10 counts from the expected 4096 triggers a warning on Serial.

1.13 Maintenance and Usability

Swapping a DUT servo takes under two minutes with no tools. The servo lifts out of the top of the mount, the horn lifts off the cross latch, and the new servo drops in. Load cells are retared at the start of each session. All wiring on the module version uses JST-XH or Dupont connectors so modules can be replaced without a soldering iron.

2. Task 2: Servo System Reliability and Safety

This section is intentionally written to be readable at a working engineering level without requiring specialist background. Think of it as what you would want to know after a few hours of googling and hands-on experience with servos.

2.1 How Servo Motors Can Fail

2.1.1 The Position Sensor Wears Out

Most hobby-grade digital servos, including the DS3218, use a small potentiometer to measure the shaft angle. A potentiometer is basically a resistive track with a wiper that slides along it. Every time the servo moves, that wiper drags along the track. Over thousands and thousands of movements, the track develops scratches, dead spots, or the conductive coating rubs off in certain areas.

When this happens, the servo's internal controller gets wrong or noisy information about where the shaft actually is. From the outside, this looks like jitter (the servo shakes back and forth around the target position), random position jumps, or the servo simply stopping in the wrong place. It gets worse gradually, which means it can be hard to catch until it becomes a real problem.

The Dynamixel XL430 avoids this completely. Its position sensor is an AS5601 magnetic encoder: a small magnet on the shaft and a chip that reads the magnetic field without any contact. No contact means no wear, which is one of the main reasons smart actuators are worth the higher cost in a production robot.

2.1.2 Running at Stall for Too Long Burns the Motor

Every motor has a winding made of copper wire wrapped in insulation. When you pass current through copper, it heats up. At normal operating conditions, the heat has time to dissipate through the servo casing. At stall, the motor is drawing maximum current continuously and generating maximum heat with no airflow to help because nothing is moving.

Digital servos do not have a thermal shutdown circuit in most cases. If you keep commanding the servo to a position it cannot reach, the internal motor keeps drawing stall current until the insulation on the windings degrades, shorting turns together. Once that happens, the motor's resistance changes, the torque drops, and the servo is effectively damaged even if it still moves. This kind of damage shows up on the jig as lower stall torque and higher no-load current than the fresh spec.

The fix in a robot's firmware is a watchdog: if the servo's position has not moved toward the

target for more than, say, 200 ms, the firmware should reduce or cut the torque command rather than letting it cook.

2.1.3 The Gears Wear and Develop Backlash

Inside a servo is a series of small gears stepping down from motor speed to output speed. The output gear is the slowest and carries the most torque, so it wears fastest. As it wears, a small gap opens between teeth that was not there originally. This gap is called backlash.

Backlash shows up as a dead zone around the target position: the servo reaches the commanded angle but can be nudged by a few degrees in either direction without the motor reacting. In a robot arm, backlash accumulates through every joint and results in positioning error that is hard to compensate in software.

The jig detects this by commanding small oscillations between two close angles and measuring the hysteresis in the encoder reading. More than about half a degree of backlash on the output shaft is a reject condition.

For high-duty-cycle food robots, metal gear sets are available as a drop-in upgrade for common servo models. They cost more but last far longer under repeated near-stall loading.

2.1.4 Electrical Noise on the Signal Wire

Standard RC servos listen for a pulse on their signal wire: the width of the pulse (between 1 ms and 2 ms) tells them the target angle. This is called PWM (pulse-width modulation). The servo's internal electronics measure exactly how long the pulse lasted, and the measured time is sensitive to any sharp electrical noise on the wire at the moment it arrives.

In a robot full of motors, heating elements, and switching power supplies, those noise sources are everywhere. A motor switching on or off creates a fast voltage spike that can travel along cables and corrupt the servo's pulse measurement, causing an unintended position step. This is especially sneaky because it happens intermittently and can look like a software bug.

The fixes are straightforward: keep signal cables away from motor power cables, use a shielded cable for servo signals, and put a small capacitor (around 100 nF) close to the servo input pin to filter out fast spikes. Switching to a bus-based protocol like the Dynamixel TTL serial eliminates the problem entirely, because a digital serial message either arrives correctly or it does not; there is no intermediate "slightly corrupted" pulse width.

2.1.5 Voltage Droop When Everything Turns On at Once

Imagine four servos in a robot all commanded to move at the same time. Each one draws its movement current from the same power rail. If the power supply or the battery cannot deliver all that current instantly, the voltage on the rail drops momentarily. This is called a droop.

Servos respond to a drooping supply in different ways: some reduce torque, some stop responding, some jerk erratically. In a shared battery system like a small robot, this is a common cause of mysterious intermittent failures that are hard to reproduce on a test bench where only one servo is powered.

The fix is to add a bulk capacitor at each servo connector, typically 470 µF to 1000 µF. This capacitor acts as a small local energy reserve that can supply the initial current spike without pulling down the common rail. It is cheap and effective.

2.2 Deep Dive: Gear Wear and PWM Noise

2.2.1 Gear Wear Under Repeated Near-Stall Loading

When you apply high torque repeatedly to a small plastic gear, the tooth surfaces experience compressive stress every time they mesh. At high torque, this stress exceeds what the plastic surface can sustain without slowly deforming. The factory grease spreads unevenly over time and dry contact in stressed areas accelerates the surface degradation.

The jig measures backlash by commanding oscillations between two closely spaced angles, 5 degrees apart in both directions, and measuring the difference between the encoder reading when approaching from the clockwise direction versus the counter-clockwise direction at the same commanded angle. This difference is the backlash. A servo with 0.1 to 0.3 degrees is healthy. Above 0.5 degrees is a yellow flag. Visible tooth damage or jumping is grounds for immediate rejection.

Metal gear servos handle this far better because the yield strength of brass or steel is an order of magnitude higher than ABS plastic. For POSHA specifically, if the same joint is near-stalling against thick dough repeatedly, metal output gear is not optional, it is a requirement.

2.2.2 PWM Noise in a Motor-Rich Environment

A brushed DC motor creates electrical noise every time one commutator segment breaks contact with the brush and another one makes it. This happens many times per shaft revolution and produces broadband noise from audio frequencies up to many tens of megahertz. In a closed metal enclosure, this noise couples into adjacent signal cables through

shared ground paths, capacitive coupling between adjacent wires, and direct induction if the signal cable runs next to the motor supply cable.

The servo input comparator is essentially a fast timer that measures how long the signal pin stays high. A noise spike that looks like a rising edge will start the timer early, and one that looks like a falling edge will stop it early, giving a pulse that is shorter or longer than intended. The servo then steps to the wrong position.

A combination of measures reduces this to a non-issue: separate cable routing with at least 20 mm between signal and power cables, a shielded cable for the servo signal with the shield grounded at the controller end only, and a 100 nF bypass capacitor on the servo input pin.

2.3 Debugging Process

When a servo misbehaves in the robot, a structured approach saves a lot of time compared to replacing things randomly.

First, isolate mechanically. Disconnect the servo from whatever it is driving so it moves freely in air. Command it through its full range. If the problem disappears in isolation, the servo itself is fine and the issue is either mechanical interference in the robot, an unexpected load, or a binding joint.

Second, if the problem appears in isolation, check the signal. If you have an oscilloscope, probe the signal pin and verify that the pulse width matches the commanded angle and that there is no noise between pulses. If you do not have an oscilloscope, try routing the signal cable away from motor power cables and see if the behaviour changes. A change confirms a noise problem.

Third, check the supply voltage under load. Command the servo to stall and measure the supply voltage at the servo connector (not at the power supply). If the voltage drops more than about 0.5 V under load, the power supply or the cable is too thin, and a bulk capacitor at the connector will help.

Fourth, if signal and power are clean, the fault is internal. Remove the servo from the robot, power it on the bench, and measure no-load current. Compare against the datasheet (DS3218 standby: about 52 mA, moving: varies with load). Elevated no-load current suggests a shorted winding turn. Then check the potentiometer or encoder by rotating the horn manually while monitoring the raw position signal. Any dead spot or discontinuity in the position output confirms internal sensor failure.

Fifth, check mechanical backlash on the jig as described in the gear wear section.

2.4 Improvement Suggestions

- **Metal gears:** Upgrade the output gear stage to brass or steel for joints that operate near stall repeatedly. Available as spare parts for common servo models.
- **Local bulk capacitor:** 470 µF at each servo connector absorbs current spikes and prevents rail droop from spreading to neighbouring servos.
- **Stall watchdog in firmware:** If position error exceeds a threshold for more than 200 ms, reduce torque command to prevent thermal damage.
- **PTC fuse on each servo power line:** A resettable fuse provides last-resort overcurrent protection without software involvement.
- **Bus-based protocol:** Moving to Dynamixel TTL serial eliminates PWM noise sensitivity entirely and adds real-time position, velocity, current, and temperature telemetry back to the controller, which makes in-field fault detection possible without any external instrumentation.

3. Task 3: POSHA System Design Improvements

3.1 Context and Background

POSHA is the kind of engineering problem that is genuinely interesting precisely because the constraints are not purely technical. The robot has to handle real food in a real kitchen, operated by people who are not engineers, cleaned between uses, and reliable enough that a meal does not get ruined by a fault. That combination of performance, hygiene, usability, and reliability is where the interesting design decisions happen.

From the website and public materials, my understanding is that POSHA automates a full meal preparation cycle: dispensing ingredients, controlling heat, stirring, and managing timing. The constraints I will focus on are reducing total cooking time and improving the physical design for both usability and maintenance.

3.2 Reducing Total Cooking Time

3.2.1 Temperature Overshoot for Smooth Phase Transitions

Many recipes involve a sequence where oil is heated to a high temperature, something is fried, and then a liquid is added that cools the vessel down before simmering begins. In a naive implementation, the heater maintains the frying temperature, the liquid is added, and then the system waits for the temperature to rise back to the simmering target.

A smarter approach is to intentionally overshoot the oil temperature slightly during the last part of the frying step. When the cold liquid hits hotter oil, the extra stored thermal energy absorbs more of the temperature shock, and the mixture reaches the simmering target faster.

The transition time between phases is reduced because the starting point for recovery is higher.

This requires a rough thermal model of the vessel (oil mass, liquid mass, specific heats, approximate heat loss rate) that the recipe scheduler uses to compute how much overshoot to apply for each transition. The hardware needed is already there: a thermocouple or RTD for the temperature reading. The cost of this feature is purely in software.

3.2.2 Pre-heating Before the User Presses Start

The largest perceived delay is waiting for a cold vessel to reach temperature. A cook pre-heats the pan before starting. POSHA can do the same: once the user loads ingredients and selects a recipe, the system can issue the preheat command immediately, before the start confirmation. By the time the user confirms, the vessel is already near the first target temperature.

This requires a recipe parser that identifies the first heating target from the instruction sequence and a confirmation prompt to let the user know the vessel is hot.

3.2.3 Overlapping Dispense and Heating

If dispensing a liquid ingredient while the vessel is still heating from the previous step, rather than waiting for the target to be confirmed first, the two phases overlap and the net time is reduced. For robust ingredients (water, stock, oil) this is always safe. For temperature-sensitive ones (cream, eggs) the overlap needs recipe-specific validation.

3.2.4 Torque-based Stirring Optimisation

Fixed-interval stirring is conservative because it assumes the worst-case sticking condition at all times. The current draw of the stirring motor is a direct proxy for how much resistance it is encountering. High current means thick or sticking food that genuinely needs stirring. Low current means free-flowing food where stirring is not necessary. Monitoring the stirring motor current and adapting the stir frequency dynamically reduces unnecessary stir cycles, cutting total cycle time and motor wear simultaneously.

3.3 Design and Functionality Improvements

3.3.1 Spice Dispenser Architecture

The spice dispenser is the most mechanically interesting problem in the system. Residue in the dispenser throat creates cross-contamination between recipes, which is both a hygiene

concern and a flavour problem. Washing between uses adds time and a water management complexity.

One approach is a single-use cartridge: a sealed compartment with a pre-metered spice charge. A linear solenoid or cam-driven pin punctures the membrane at the base and the charge falls into the vessel. There is no shared path between cartridges, so cross-contamination and the wash cycle both disappear. Complexity moves into the cartridge supply chain, but the robot's internal mechanism becomes very simple.

For continuously variable amounts, a vibratory feeder over a precision load cell below the vessel gives closed-loop mass dispensing: the feeder runs until the load cell confirms the target mass has been added. This is standard practice in pharmaceutical and food manufacturing. The hopper uses a removable flexible liner for cleaning between different spices.

3.3.2 Accessibility for the End User

All user-facing interfaces should be between hip and chest height: spice cartridge swaps, water inlet, and any surface needing regular cleaning. A front-loading drawer for the spice module, similar to a printer paper tray, allows cartridge swaps at waist height. The latch should click positively when seated so the user does not need to check an indicator.

A modular split-unit architecture, where the computation and power electronics are in a separate unit that can be shelf-mounted, reduces the weight of the cooking module to a more accessible form factor. The cooking module becomes easier to clean and to position. This is the same principle as a split air conditioning system or a separate espresso boiler: the heavy electrical hardware goes somewhere convenient and the user-facing hardware goes where it needs to be.

Connecting the water inlet directly to a tap or an inline purifier rather than requiring manual tank fills removes one routine maintenance step entirely.

3.3.3 Adaptive Recipe Learning

A self-improving mode where the system logs the actual time to reach each thermal milestone during a cook, compares it against the recipe's nominal timing, and adjusts preheat and overshoot parameters for subsequent runs would progressively tighten the timing accuracy. After three or four runs of the same recipe, the system has a thermal model specific to that recipe and that kitchen (ambient temperature, altitude, vessel condition). This is a simple adaptive controller and can run on an embedded microcontroller without cloud connectivity.